SYSTEM DESIGN INTERVIEW

Design Gmail

large-scale email service: SMTP ingest, IMAP/web delivery, threading, search, spam, labels

~ 45 minute read

What's Inside

01 Problem Statement

Requirements and scope

Design an email service like **Gmail**. The system must ingest mail over SMTP, deliver via IMAP / POP / web / mobile, group messages into **conversations**, support full-text search, filter spam at scale, and keep ~25 GB per user reliably for billions of users.

Clarifying Questions

Question	Answer
Users?	~2 billion active mailboxes; global
Volume?	~300 billion emails/day inbound (most non-human)
Storage/user?	~25 GB free tier; petabyte-scale aggregate
Protocols?	SMTP in/out, IMAP, POP3, JMAP, web, mobile push
Threading?	Conversation view by Subject + In-Reply-To/References
Search?	Full-text, near-real-time, per-user partition
Spam?	ML-scored at SMTP edge; user feedback loop
Attachments?	Up to 25 MB; deduped via content hash

02 Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Receive mail	Accept SMTP on MX; verify DKIM/SPF/DMARC; spam-score; deliver
Send mail	Compose, queue, retry, DSN handling; outbound SMTP relay
Read mail	IMAP/POP/web/mobile; folders/labels; mark-read state
Threading	Group messages by conversation (canonical thread_id)
Labels	Tag messages with M:N labels; system + user labels
Search	Full-text query: from/to/subject/body/has:attachment/label:
Spam	Auto-classify; user can mark spam / not-spam
Attachments	Store, dedup by content-hash, virus-scan

Non-Functional Requirements

Requirement	Target
Availability	99.99% mailbox; 99.9% search index
Durability	11 nines; 3+ replicas; cross-region for hot data
Latency	<200 ms inbox load; <500 ms search
Ingest	~3.5 M msgs/sec peak; 99% within 30 s end-to-end
Storage	~25 GB/user; multi-EB aggregate
Consistency	Strong per-mailbox; eventual for search index
Spam recall	≥99.9% caught; <0.1% false positive

03 Capacity Estimation

Math for scale

Traffic

- Active users: **~2 billion**
- Emails inbound: **~300 billion / day** (Gmail-public-figure ballpark)
- Average rate: 300B / 86,400 s ≈ **3.47 M msgs/sec**
- Peak rate: ~2× average ≈ **~7 M msgs/sec**
- Composition: ~85% bulk/automated/spam, ~15% human-to-human
- Outbound: ~10% of inbound (most users read more than they send)

Storage

- Average kept message: ~75 KB body + ~50 KB headers/metadata + occasional attachment
- Per user: ~25 GB usable (free tier; paid tiers up to 2 TB)
- Aggregate: 2B × 25 GB = **~50 EB** raw
- After dedup, compression, tiering: **~10–15 EB** live + cold archive
- Daily growth: 300B msgs × 100 KB avg = **~30 PB / day** raw
- After spam filtering (~85% dropped or kept compactly): ~3–5 PB/day stored

Search Index

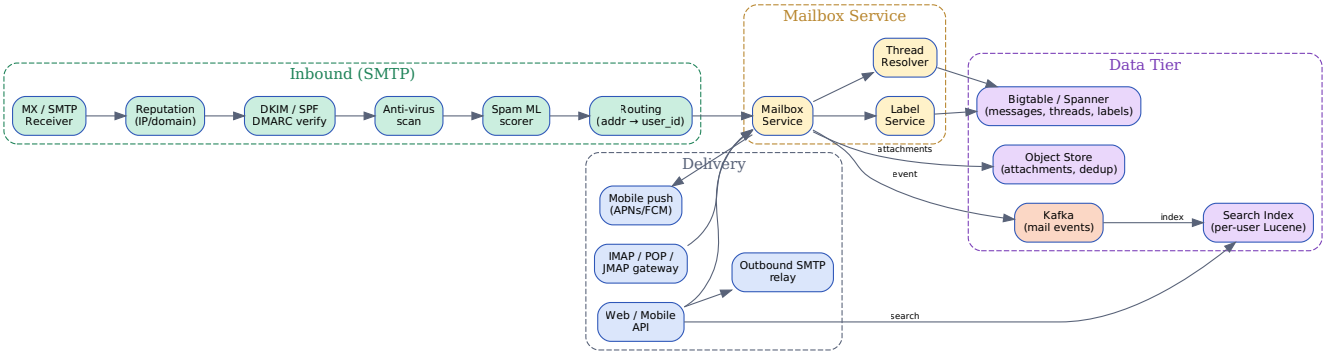
- ~15% of messages searched per day → ~45 B query-relevant docs touched
- Index size: ~30% of raw text size with posting lists ≈ **multi-PB**
- Refresh latency target: **<30 seconds** from delivery to searchable
- QPS: ~1–2 M searches/sec peak (auto-complete + manual queries)

> KEY
>
> **Key Numbers**
>
> **2 B** users · **300 B** msgs/day · **3.5 M/sec** ingest · **25 GB**/user · **~10 EB** live storage · **<30 s** search freshness.

04 High-Level Architecture

System overview

The architecture has three planes: an **ingest plane** that accepts SMTP and runs reputation, authentication, anti-virus, and spam scoring; a **storage plane** built on Bigtable/Spanner plus a content-addressed object store for attachments; and a **delivery plane** that exposes IMAP, POP, JMAP, web, and mobile push. Search runs as a separate per-user inverted-index service fed by a Kafka event stream.



Inbound SMTP runs through reputation/auth/AV/spam, then writes to the per-user mailbox; delivery and search read from that store.

Architecture Highlights

- **MX / SMTP receivers:** globally distributed; accept on port 25; speak ESMTP, STARTTLS
- **Reputation:** IP/domain block-lists, greylisting, RBL checks; reject before parsing body
- **DKIM/SPF/DMARC:** cryptographic and policy auth — *essential for spam decisions*
- **Anti-virus:** ClamAV-class scanners on attachments and embedded scripts
- **Spam scorer:** LightGBM/DNN model returning [0..1]; threshold splits inbox vs spam
- **Mailbox Service:** the source of truth API for read/write; routes by user_id
- **Bigtable/Spanner:** primary store; row key (user_id, thread_id, msg_id)
- **Object Store:** attachments by content-hash; dedup across all users
- **Search Index:** per-user Lucene shard, updated via Kafka events
- **Delivery:** IMAP/POP/JMAP gateways, web/mobile API, FCM/APNs push, outbound SMTP

05 SMTP Ingest Pipeline

From MX to mailbox

End-to-End Pipeline

1. Sender's MTA opens TCP/25 to our MX, advertises EHLO, STARTTLS
2. **Reputation check:** source IP / sending-domain reputation (IP4R, RBLs); reject 5xx for known-bad
3. **Greylisting (light):** defer first delivery from unknown sender; legitimate MTAs retry
4. **SPF:** verify SMTP MAIL FROM domain authorizes the source IP
5. **DKIM:** verify body+selected-headers signature against the sender's published public key
6. **DMARC:** apply alignment policy; quarantine/reject on policy failure
7. **Anti-virus scan:** attachments and inline scripts; quarantine if positive
8. **Content extraction:** parse MIME tree; extract headers, text, attachments
9. **Spam scoring:** ML model returns score; ≥0.5 → Spam label, ≥0.95 → silent drop
10. **Routing:** resolve to_addr → user_id (account directory); handle aliases, +tags, mailing-lists
11. **Thread resolution:** compute thread_id from In-Reply-To / References / Subject
12. **Persist:** write row to Bigtable; attachments to object store keyed by SHA-256
13. **Emit event:** publish to Kafka for indexing, push notification, audit log
14. **Acknowledge:** 250 OK to sender — only after durable persistence

Authentication Decision Matrix

SPF	DKIM	DMARC	Action
pass	pass	pass	Inbox (subject to spam score)
fail	pass	pass (DKIM aligns)	Inbox; lower trust
pass	fail	pass (SPF aligns)	Inbox; lower trust
fail	fail	p=quarantine	Spam folder
fail	fail	p=reject	Reject 550 at SMTP
—	—	no record	Treat as suspicious; weight on spam model

TIP

Reject Early, Save Money

> Most inbound is junk. Rejecting at the SMTP envelope (before DATA) on reputation alone discards the majority of bytes — saving bandwidth, AV cycles, and storage. Anti-virus and ML scoring run only after the envelope passes.

06 Storage Model

Bigtable, Spanner, and dedup

We store messages in a wide-column store such as **Bigtable** (or Spanner where strong cross-row consistency is needed). The row key embeds `(user_id, thread_id, msg_id)` so all messages in a conversation live next to each other on disk — a single range scan loads the entire thread.

Bigtable Row Layout

```
# Row key: <user_id>/<thread_id>/<msg_id>
#   user_id is a 64-bit hash → balanced regions
#   thread_id sorts within user → conversation locality
#   msg_id is monotonic (timestamp-derived)

row_key = f"{user_id:016x}/{thread_id:016x}/{msg_id:016x}"

# Column families:
#   meta:    headers, dates, sender, subject, snippet
#   body:    text/plain, text/html (compressed)
#   labels:  one column per label_id, value=timestamp
#   attach:  one column per attachment, value=content_hash
#   flags:   read/unread, starred, important, draft

# Example cells:
#   meta:from        "alice@x.com"
#   meta:subject     "Re: Q3 plans"
#   labels:INBOX     1714972800
#   labels:STARRED   1714972900
#   attach:f4ab12..  "deck.pdf|application/pdf|2.4MB"
#   body:html        <gzipped html>
```

Why This Row Key

- **user_id prefix:** all of a user's data is one contiguous range — easy GDPR delete
- **thread_id next:** conversation messages co-locate; one scan loads the full thread
- **msg_id last:** within a thread, ordered by arrival; cursor pagination is trivial
- **Hash user_id first:** avoids hot regions when popular tenants ramp
- **Tablet split:** Bigtable auto-splits ranges; very large mailboxes fan across tablets

Attachment Deduplication

- Compute **SHA-256** of the attachment bytes during MIME parse

- Object-store key = content hash; PUT is idempotent (same hash → same blob)
- Reference count tracked separately (or rely on lifecycle rules for cleanup)
- **Win:** a viral PDF emailed to 1 M users stores *once*, ~1 M tiny references
- **Caveat:** per-user encryption breaks dedup — solved with envelope encryption (per-blob key wrapped per-user)
- Estimated savings: **30–40%** of attachment bytes on real-world workloads

Bigtable vs Spanner

Aspect	Bigtable	Spanner
Consistency	Single-row atomic; eventual across rows	External consistency, multi-row TX
Throughput	Massive (sequential row writes)	Lower; TX coordination cost
Use	Message rows, label cells	Account directory, billing, quota
Cost	$ per node + storage	$$ — pricier per QPS
Schema	Schemaless cells	Strong schema with secondary indexes

07 Threading (Conversation View)

Grouping messages

Gmail's signature feature is the **conversation view**: a back-and-forth thread shown as one expandable card. Threading is computed at delivery time and stored as a canonical `thread_id` per user. Clients never re-thread.

Threading Algorithm (RFC 5322 + Gmail tweaks)

```
def resolve_thread_id(user_id, msg):
    # 1. Prefer the chain referenced by the message itself.
    candidates = []
    if msg.in_reply_to:
        candidates.append(msg.in_reply_to)
    candidates.extend(reversed(msg.references or []))  # newest-first

    for ref in candidates:
        existing = lookup_by_message_id(user_id, ref)
        if existing:
            return existing.thread_id

    # 2. Fallback: subject normalisation + sliding window.
    norm = normalise_subject(msg.subject)
    #   strip 'Re:', 'Fwd:', '[list]', trailing whitespace, case
    if norm:
        recent = recent_threads_with_subject(user_id, norm,
                                             window=timedelta(days=14))
        # require at least one shared participant for safety
        for t in recent:
            if msg.participants & t.participants:
                return t.thread_id

    # 3. New thread.
    return new_thread_id()

def normalise_subject(s):
    s = re.sub(r'^(re|fwd|fw|aw)\s*:\s*', '', s, flags=re.I)
    s = re.sub(r'^\[.*?\]\s*', '', s)  # strip mailing-list tags
    return s.strip().lower()
```

Why Per-User Thread IDs

- **Privacy:** Alice's view of a thread shouldn't leak Bob's participation
- **Asymmetric membership:** Bob may be added later; his thread starts mid-conversation

- **BCC handling:** hidden recipients shouldn't change the visible thread for others
- **Message-ID is global, but threading is local** — dual lookup tables: by msg-id, by thread-id

Edge Cases

Case	Behaviour
Missing References header	Fall back to subject + participants window
Subject changed mid-thread	References still chain; thread persists
Common subject ("hi")	Subject heuristic disabled below a length/uniqueness floor
Mailing list bounces	Strip [list-tag] and re-evaluate
Forwarded standalone	User explicitly forwarded → start new thread
Self-reply (sent + received)	Same thread on both sides via cross-ref index

INSIGHT

Why Subject Alone Is Not Enough

Two unrelated people can both email Alice with subject `Lunch?`. Subject matching without participant overlap or a References chain produces incorrect merges. Always require a corroborating signal.

08 Labels vs Folders

Gmail's tag model on top of IMAP

Traditional mail clients organise mail in **folders**: a tree where each message lives in exactly one node (1:N). Gmail uses **labels**: a tag set where each message can carry many labels (M:N). The IMAP protocol only knows about folders, so Gmail *projects* labels onto folders for IMAP clients.

Labels vs Folders

Property	Folders (IMAP)	Labels (Gmail)
Cardinality	1 message → 1 folder	1 message → many labels
Structure	Hierarchical tree	Flat or nested namespace
Storage	Move = copy + delete	Add/remove a column cell
Queries	List by folder	Intersect/union of labels
Archive	Move to Archive folder	Remove INBOX label
Trash	Move to Trash folder	Add TRASH label, remove others
IMAP fidelity	Native	Each label appears as a virtual folder

Schema for Labels

```
-- Logical model (the real impl is Bigtable cells, but this captures it)
CREATE TABLE labels (
    user_id     BIGINT,
    label_id    BIGINT,
    name        VARCHAR(120),
    parent_id   BIGINT,        -- nested labels
    color       VARCHAR(16),
    is_system   BOOLEAN,       -- INBOX, SENT, SPAM, TRASH, STARRED, IMPORTANT
    PRIMARY KEY (user_id, label_id)
);

CREATE TABLE message_labels (
    user_id     BIGINT,
    msg_id      BIGINT,
    label_id    BIGINT,
    applied_at  TIMESTAMP,
    PRIMARY KEY (user_id, msg_id, label_id),
```

```
    INDEX (user_id, label_id, applied_at DESC)  -- list a label
);
```

Mapping Labels Onto IMAP

- **Each label = one IMAP folder** (e.g., `[Gmail]/All Mail`, `[Gmail]/Sent`)
- A message with N labels appears in N IMAP folders — same UID across them
- IMAP *MOVE* from `INBOX` → `Receipts` = remove INBOX label, add Receipts label
- IMAP *COPY* = add a label
- `[Gmail]/All Mail` is the universe; deleting from a label folder doesn't remove from All Mail
- Gmail-specific extensions (`X-GM-LABELS`, `X-GM-THRID`) expose native model to IMAP-aware clients

09 Search

Per-user inverted index

Search must return results in <500 ms across a 25 GB mailbox, with <30 s freshness from delivery. The natural partition key is `user_id`: queries always run against a single user's corpus.

Index Architecture

- Per-user **Lucene**-style inverted index, sharded by user_id
- Many users per shard (small users co-tenant); whales get dedicated shards
- Fields: `from`, `to`, `cc`, `subject`, `body`, `label`, `has:attachment`, `filename`, `before:`, `after:`
- Tokenise body with language detection; ICU analyser for CJK / RTL
- Store snippets in the index for instant preview without a Bigtable round-trip
- Replicate index 3-way; queries hit any replica (read-any-quorum)

Near-Real-Time Refresh

1. Mailbox Service publishes a delivery event to Kafka topic `mail.events`
2. Indexer consumer (per shard) reads events partitioned by user_id
3. Builds in-memory segment; flushes to disk every 5 seconds (or 10 MB)
4. Searcher opens new segment via Lucene's `SearcherManager.maybeRefresh()`
5. Net visibility: typically **5–15 seconds**; SLO <30 s
6. Background merger compacts small segments hourly; major merge nightly

Inverted Index vs Bigtable Scan

Approach	Pros	Cons
Bigtable scan + filter	No second store; consistent with mailbox	Slow on large mailboxes; reads every byte of body; no relevance ranking
Dedicated Lucene index	Sub-second queries; scoring; phrase/wildcard	Extra storage (~30%); index lag; rebuild cost on schema change
Hybrid: index for text, BT for fields	Best of both; cheap header filters	Two query paths; merge complexity

> **TIP**
>
> **Always Pin to user_id**

> Every query carries the authenticated user_id as the first filter. The index is partitioned by user_id, so a query touches exactly one shard. This is what makes <500 ms feasible across a 25 GB mailbox.

10 Spam Filtering

ML at SMTP edge with feedback loop

Multi-Layer Defence

- **L1 — IP/domain reputation:** reject at SMTP envelope; cheapest layer
- **L2 — Authentication:** SPF/DKIM/DMARC; failures mark suspicious
- **L3 — Content fingerprints:** known-bad URL/hash blocklists
- **L4 — ML model:** LightGBM / DNN scoring full message
- **L5 — User feedback:** 'Mark as spam' / 'Not spam' close the loop

ML Features

- Sender reputation history (per IP / per domain / per ASN)
- Header anomalies (date skew, malformed Message-ID, mismatched From/Reply-To)
- Body features: TF-IDF, URL count, image-only ratio, language match
- Recipient features: how the user has historically treated this sender
- Content embeddings (transformer-based) for novel-spam generalisation
- Engagement signal: how often this sender's mail gets opened / replied to

Decision Thresholds

Score	Action
< 0.10	Inbox; high-confidence ham
0.10 – 0.50	Inbox; light category sort (Promotions / Updates / Social)
0.50 – 0.95	Spam folder; user can recover
≥ 0.95	Silent drop / 5xx reject (clear malware/phish)

Feedback Loop

1. User marks a message as spam (or not-spam)
2. Event written to `spam.feedback` Kafka topic
3. Online learner updates per-user weights immediately (personalisation)
4. Aggregated daily into the global model retraining set
5. Promote new model after offline AUC + canary traffic ≥ baseline

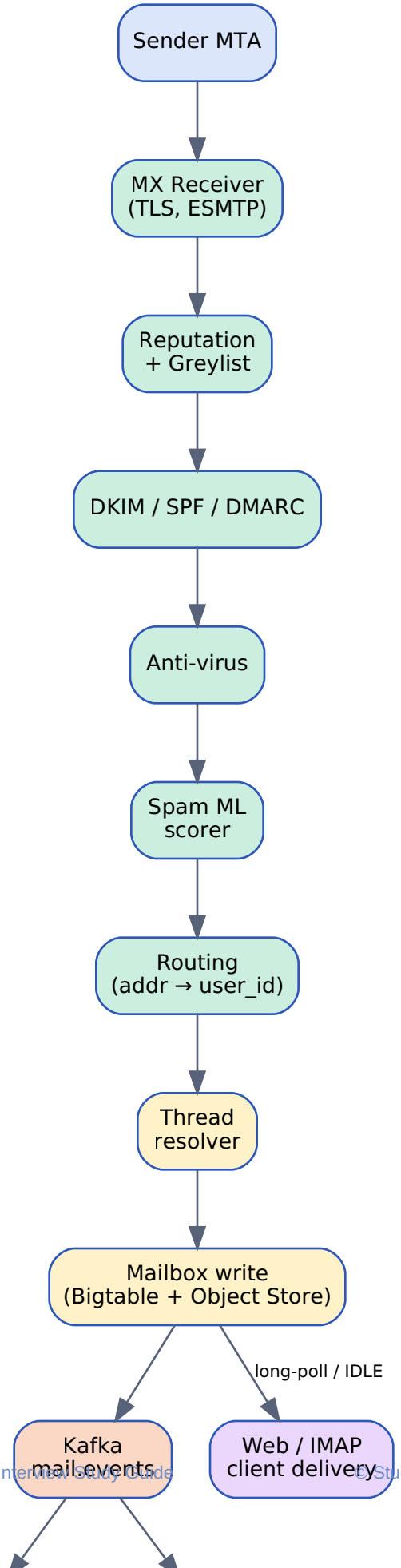
WARNING

Model Drift

Spammers retool weekly. Without continuous retraining and drift monitoring, recall drops within days. Track per-day false-positive and false-negative rates; alert on >0.1% FP for human-to-human mail.

11 Email Lifecycle

From sender to notification



End-to-end lifecycle: SMTP receive → ingest pipeline → mailbox write → search index update → mobile push.

Latency Budget (target: <30 s end-to-end)

Stage	Typical	P99
MX accept + TLS	30 ms	150 ms
Reputation + auth	20 ms	200 ms
Anti-virus scan	100 ms	1 s
Spam ML score	50 ms	300 ms
Thread resolve	20 ms	200 ms
Mailbox persist	50 ms	500 ms
Kafka publish	10 ms	100 ms
Indexer flush	5 s	30 s
Push to device	1 s	10 s
Total visible	**< 7 s**	**< 30 s**

12 Outbound Send Path

Compose to remote MX

Send Steps

1. Web/mobile composer POSTs to API: `POST /v1/messages`
2. Server validates auth, quota, recipient list size
3. Persist as `DRAFT` in user's mailbox first (so a crash doesn't lose it)
4. On Send: write `SENT` label, enqueue to outbound queue
5. Outbound MTA looks up MX records for each recipient domain
6. DKIM-sign the outgoing message with our domain's selector
7. Connect to remote MX over TLS; deliver via SMTP
8. On 5xx: bounce → DSN to sender; on 4xx: retry with backoff (15 min, 1 h, 4 h, 24 h)
9. Final result (delivered or bounced) updates the SENT message status

Outbound Reputation Hygiene

- **Per-IP warm-up** for new sending IPs to avoid block-listing
- **Bounce-rate monitoring:** auto-disable accounts >5% hard-bounce
- **Spam-trap detection:** if user's outbound triggers RBL, suspend
- **Rate-limit per user:** ~500 recipients/day for free accounts
- **Feedback loops:** ARF reports from large providers feed user-account abuse model

> **TIP**
>
> **Persist Drafts Before Send**
>
> Mobile networks drop. Persist drafts on every keystroke (debounced) so Send is just a flag flip. The Sent message and the Inbox copy at the recipient should appear within 5 seconds of clicking Send.

13 Protocols & Comparison

IMAP, POP, JMAP, SMTP

Protocol Roles

Protocol	Direction	Role
SMTP	in / out	Server-to-server transport (port 25, 587 submit)
IMAP	client read	Stateful sync; folders; flags; partial fetch
POP3	client read	Download-and-delete; minimal state on server
JMAP	client read	Modern JSON-over-HTTPS; push, batch, rich queries
HTTPS API	client read/write	First-party web/mobile; richest features
DKIM/SPF/DMARC	auth	Domain-level message authentication

SMTP vs JMAP

Aspect	SMTP / IMAP	JMAP
Transport	Stateful TCP, multi-port	HTTPS, single port
Encoding	RFC 5322 / MIME	JSON
Push	IMAP IDLE (poll-ish)	Native push channel
Batching	Round-trip per command	Single batched call
Search	Server-side, limited	Rich filter + sort
Mobile fit	Battery-hostile	HTTP-friendly

Provider Comparison

Provider	Org Model	Search	E2E Encryption	Notable
Gmail	Labels (M:N) on IMAP folder façade	Per-user Lucene; conversations	Optional S/MIME, no E2E by default	ML spam, conversation view, AI features

Provider	Org Model	Search	E2E Encryption	Notable
Outlook (M365)	Folders + Categories	Exchange Search (Lucene-based)	S/MIME, optional Purview encryption	Tight Office/Teams integration
ProtonMail	Folders + Labels	Client-side encrypted index	Native E2E (PGP)	Privacy-first; smaller scale

INSIGHT

Why Gmail Keeps IMAP

JMAP is technically nicer, but two decades of MUAs, scripts, and devices speak IMAP. The value of an interoperable inbox dominates the cost of translating labels to folders. Gmail-native clients use the HTTPS API.

14 Failure Modes

What can go wrong

Failure	Impact	Detection	Mitigation
Bigtable hot tablet (whale user)	Mailbox slow; queue backs up	Tablet QPS / latency dashboard	Salt user_id; shard whale to dedicated cluster
DKIM verification slow / DNS flaky	Ingest latency spikes	P99 verify time alarm	Async DKIM: provisionally accept, re-verify offline; cache pubkeys
Spam model drift	Recall drops; complaints rise	Daily FP/FN metric, user-mark-spam rate	Continuous retraining; canary rollout; per-user personalisation
MX overload (mail storm)	Senders see 4xx; queue grows	Conn-rate, queue-depth alarm	Per-source rate limit; tarpit; auto-scale receivers
Index lag > 30 s	Search misses recent mail	Refresh-lag SLO breach	Add indexer consumers; flush more often; degrade to BT scan
Object store unavailable	Attachments fail to load	PUT/GET error rate	Multi-region replication; serve stale via CDN; degrade attachment-only
Outbound IP blocklisted	Mail to that domain bounces	Bounce-rate per IP	Reputation pool: rotate sending IP; submit delisting
AV scanner crash loop	Ingest stalls (fail-closed)	Worker liveness	Bypass with quarantine label and async re-scan
Cross-region partition	Cannot replicate	Replication lag alarm	Continue local writes; reconcile on heal

WARNING

Fail-Closed on Auth, Fail-Open on Convenience

If anti-virus is down, hold mail (fail closed) — a malware delivery is worse than a delay. If the search index is lagging, allow inbox load (fail open) — search can be stale, but mail must arrive.

15 Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Search freshness vs cost	5–15 s typical; ≤30 s SLO	Tighter freshness multiplies indexer cost (more flushes, more replicas). 30 s is the user-perceptible sweet spot.
Dedup at storage vs query time	Storage-time content-hash	Hash work on ingest is a fixed cost; saves 30–40% of attachment bytes forever. Query-time dedup wastes storage and complicates retention.
IMAP fidelity vs Gmail-native model	Both: labels-as-folders + X-GM extensions	Native model is richer; IMAP keeps two decades of clients working. Worth the translation cost.
Threading at delivery vs at read	At delivery (canonical thread_id)	Heavier write path; constant-time read. Re-threading on read would double inbox-load cost.
Spam: silent drop vs spam folder	Drop only at ≥0.95; otherwise spam folder	Silent drop hides false positives forever. Spam folder lets users recover at the cost of clutter.
Bigtable vs Spanner	Bigtable for messages, Spanner for accounts	Bigtable is cheaper at scale; Spanner gives ACID where it matters (billing, quota, login).
Per-user index shard vs global	Per-user partitioned (many users / shard)	Per-user keeps query latency tight; global index would dominate compute and complicate ACLs.
E2E encryption	Off by default	E2E breaks search, spam scoring, and dedup. Provider-side encryption + audit is the practical compromise.

16 Interview Playbook

How to present this

Gmail is a deceptively deep prompt: SMTP ingest, threading, labels, search, and spam are each a system in themselves. Drive the discussion; do not let the interviewer pull you into a single rabbit-hole until the skeleton is up.

45-Minute Outline

1. **Clarify (3 min):** 2 B users, 300 B msgs/day, 25 GB/user; protocols (SMTP/IMAP/web)
2. **Capacity (4 min):** 3.5 M/sec ingest, ~10 EB live, ~30 PB/day raw, search lag <30 s
3. **Architecture (5 min):** SMTP MX → reputation/auth/AV/spam → mailbox → BT/object/index/Kafka → IMAP/web/push
4. **Ingest deep dive (8 min):** SPF/DKIM/DMARC matrix, threading, 250-OK only after persist
5. **Storage (6 min):** Bigtable row key (user, thread, msg); attachment dedup by SHA-256
6. **Threading (5 min):** References chain → subject + participant fallback; per-user thread_id
7. **Search (5 min):** per-user Lucene shard, Kafka-driven NRT refresh, 5–15 s freshness
8. **Spam (4 min):** 5-layer defence, ML at scoring, user feedback closes the loop
9. **Failures (3 min):** hot user, model drift, index lag, fail-closed on AV
10. **Trade-offs (2 min):** Bigtable/Spanner split; labels-as-folders for IMAP

Numbers To Memorise

- **2 B** users · **300 B** msgs/day · **3.5 M** msgs/sec
- **25 GB**/user · **~10 EB** live · **~30 PB/day** raw inbound
- Search freshness **< 30 s**, query **< 500 ms**
- Inbox load **< 200 ms**; total visible delivery **< 7 s**
- Spam: ≥**99.9%** recall, <**0.1%** false positive on human mail
- Attachment dedup saves **30–40%** of bytes

Common Follow-ups & Answers

- **Q: How do you scale a 'whale' mailbox?** A: Hash user_id into row key; Bigtable auto-splits tablets; very large mailboxes get a dedicated cluster and a sharded index.
- **Q: How does conversation view stay consistent across IMAP and web?** A: thread_id is computed once at delivery and stored on each message; IMAP exposes via `X-GM-THRID`; web reads the same column.

- **Q: How do you keep search fresh?** A: Mailbox publishes to Kafka; per-user-shard indexer flushes 5–10 s segments; Lucene `maybeRefresh` opens them. SLO <30 s.
- **Q: Why labels and not folders?** A: Real workflows are M:N (a receipt is also Travel and also Important). Labels are cheap cell adds in Bigtable; moves between folders would be a copy + delete.
- **Q: How do you stop spam loops?** A: SPF/DKIM/DMARC at the door, per-IP/domain reputation, ML score, user feedback into per-user weights, daily global retrain with canary.
- **Q: GDPR delete?** A: All of a user's rows share the user_id prefix; one range delete in Bigtable plus an attachment-refcount sweep handles it.

> KEY
>
> **One-Sentence Pitch**
>
> Receive over SMTP with reputation + DKIM/SPF/DMARC + AV + ML spam → persist to a Bigtable row keyed by `(user_id, thread_id, msg_id)` with attachments deduped in object storage → fan a Kafka event to a per-user Lucene index → serve to IMAP/JMAP/web with conversation view computed at delivery.